

# **Data Pipeline Blueprint**

## **Due scenari TO BE per progetti data-intensive**

Documento di discussione per CEO, CTO, Gianfranco e Tech Committee

30 giugno 2026



## Contesto

### **Tre casi diversi, stesso tema di governo del dato**

ProSIGNAL, Kiron CDG e CDG interno partono da fonti e obiettivi diversi. Il punto comune è governare dati, regole, anomalie e output in modo spiegabile, non produrre solo una dashboard in più.

## Esigenza

### **Serve una blueprint, non una soluzione rigida**

Senza un modello condiviso, ogni progetto rischia di ricostruire da zero tracciati, mapping, controlli, log e gestione degli scarti. La blueprint deve lasciare flessibilità sui casi specifici, ma standardizzare qualità, auditabilità e costi di gestione.

## Obiettivi

### **La decisione riguarda il modello operativo riutilizzabile**

Il deck mette a confronto due ipotesi TO BE, Dagster/dbt/Metabase su AWS e Talend/Qlik, chiarendo cosa resta comune, cosa cambia davvero e quali verifiche servono prima di applicare la blueprint a ProSIGNAL e Kiron.

# La blueprint separa le responsabilità del dato, dall'ingresso fino all'uso business.

Il valore non sta in un singolo tool. Sta nel rendere chiaro dove nasce il dato, quali regole lo trasformano, quali controlli lo validano e quali output possono essere usati con fiducia.



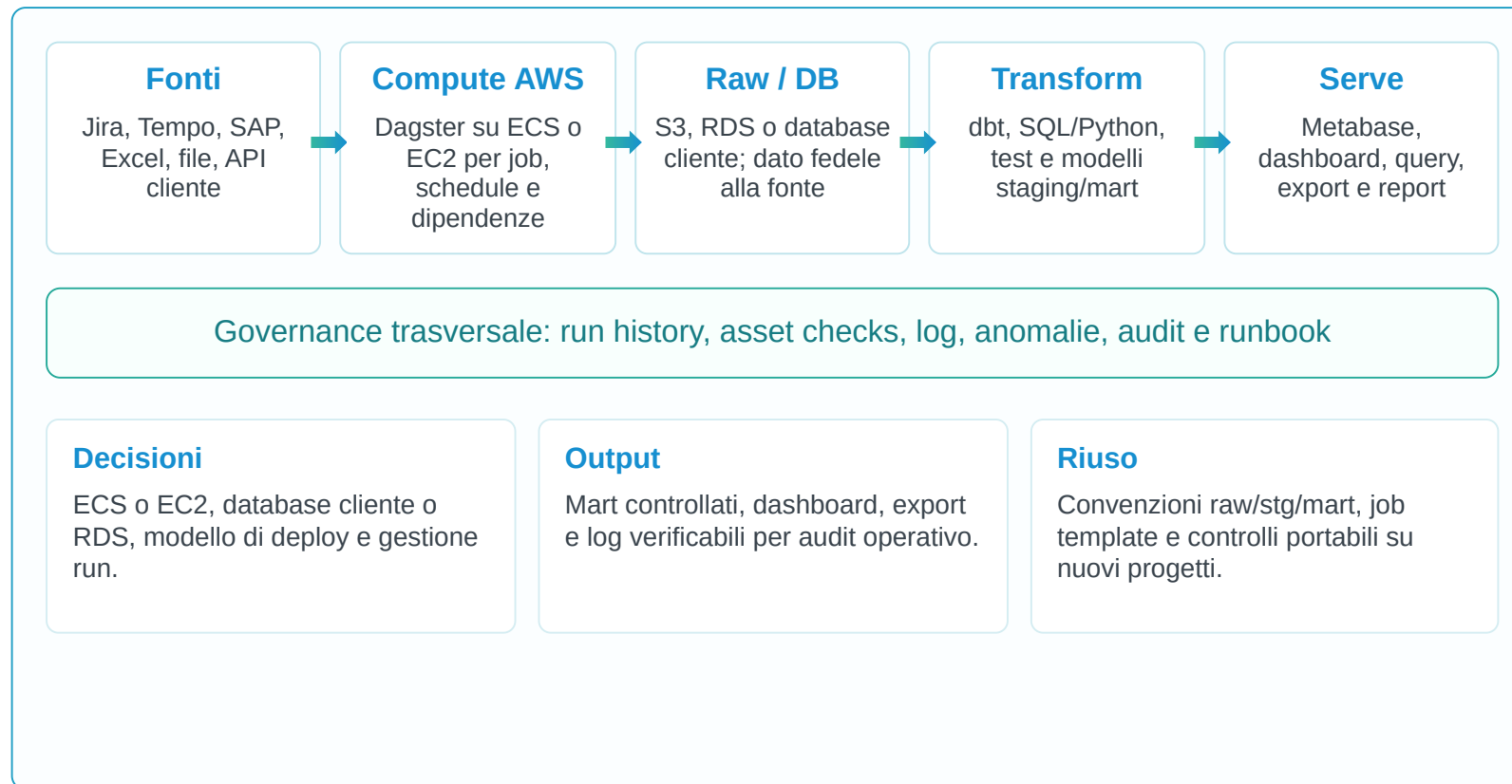
Una pipeline riusabile nasce quando componenti comuni e adattamenti specifici sono separati fin dall'inizio.

# I due scenari vanno confrontati sul modello operativo, non solo sulla lista dei prodotti.

Le parti comuni devono rimanere stabili. Cambia il modo in cui vengono implementate, governate e portate in esercizio.

| Parti comuni                                                                                                                                  | Scenario A   AWS                                                                                                                                                                   | Scenario B   Qlik                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Data contract, quality gate, mart, output, run log e responsabilità operative.</p> <p>Questa è la base da riusare sui progetti futuri.</p> | <p>Dagster coordina job e dipendenze, dbt rende esplicite trasformazioni e test, Metabase espone mart e dashboard.</p> <p>È lo scenario con maggiore controllo ingegneristico.</p> | <p>Talend/Qlik presidia integrazione, quality e analytics in una filiera più product-oriented.</p> <p>Licenze, tenant e capacity diventano parte della decisione.</p> |

# Lo scenario AWS separa orchestrazione, trasformazioni, storage e presentation layer.



## Come leggerlo

Dagster governa il flusso: quando parte, cosa dipende da cosa, dove si è fermato e quali controlli hanno dato esito negativo.

dbt separa le regole di trasformazione dalla parte di ingestion. Metabase consuma output già controllati: non diventa il motore dati.

ECS o EC2 restano una scelta di costo e operation. RDS o DB cliente dipende dal contesto del progetto.

# Nello scenario AWS ogni componente resta esplicita e governabile.

| Componente   | Come viene soddisfatta                                           | Impatto per il comitato                                                       |
|--------------|------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Ingestion    | Job Dagster su ECS/EC2, connettori, upload file, API e schedule. | Maggiore controllo su run, retry, volumi e responsabilità tecniche.           |
| Preparazione | Parsing, mapping e normalizzazione in SQL/Python e modelli dbt.  | Regole leggibili, versionabili e riusabili tra progetti simili.               |
| Qualità      | Test dbt, asset checks, scarti, quadrature e run history.        | Errori e anomalie diventano verificabili, non dipendono da controlli manuali. |
| Esposizione  | Mart e dataset BI-ready esposti via Metabase, export o query.    | Il layer BI resta leggero e consuma dati già controllati.                     |

# Lo scenario AWS è forte quando servono controllo, logiche custom e riuso tecnico.

## Riuso

Alto se asset Dagster, modelli dbt e convenzioni raw/stg/mart diventano standard.

## Scalabilità

Buona, legata a disegno cloud, database, scheduling e separazione dei layer.

## Costi

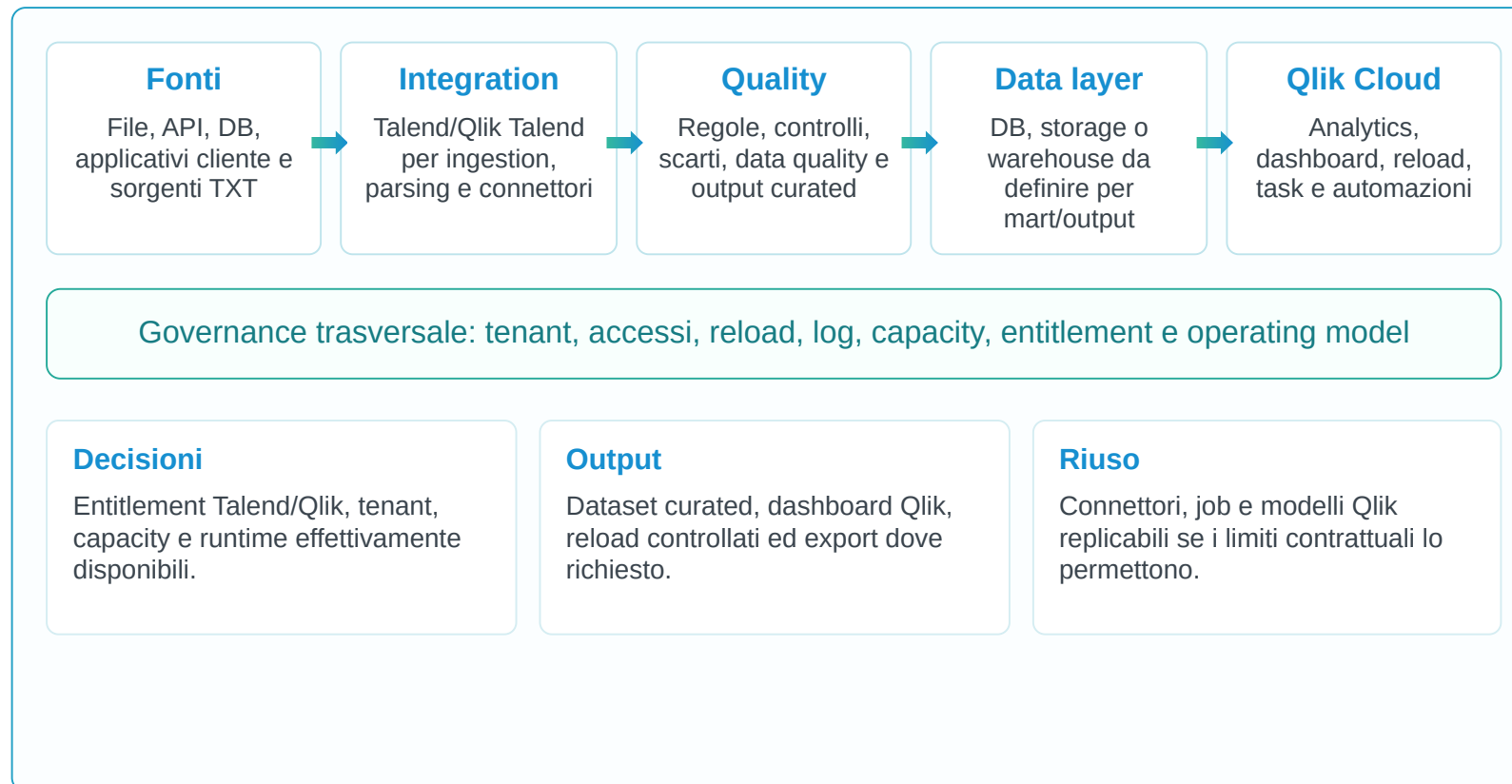
Licenze potenzialmente leggere; pesano infrastruttura, run e competenze.

## Blocker

Skill data engineering, operation AWS e requisiti BI dei clienti esterni.

Fit naturale per file processing custom, controlli specifici e reference implementation CDG.

# Lo scenario Talend/Qlik concentra integrazione, quality e analytics in una filiera più gestita.



## Come leggerlo

Talend/Qlik sposta parte del lavoro custom verso una piattaforma di integrazione e quality.

Qlik Cloud diventa il layer principale per consumo, dashboard e analytics, con automazioni operative a supporto.

Il punto da verificare è concreto: entitlement, capacity e fit su file grandi, fixed-column e controlli cross-file.



Nello scenario Qlik/Talend la piattaforma diventa parte del modello operativo.

| Componente   | Come viene soddisfatta                                                               | Impatto per il comitato                                                            |
|--------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Ingestion    | Connettori e job Talend/Qlik Talend, con runtime e tenant da confermare.             | Avvio più rapido se licenze, ambienti e connettori sono pronti.                    |
| Preparazione | Data integration, regole di mapping e trasformazioni gestite nella filiera prodotto. | Più standardizzazione, ma maggiore dipendenza dal perimetro della piattaforma.     |
| Qualità      | Data quality, scarti, controlli e monitoraggio secondo capability abilitate.         | Va verificato cosa è incluso nel contratto e con quali limiti di capacity.         |
| Esposizione  | Qlik Cloud per dashboard, analytics, reload, task chain ed eventuali export.         | Il layer di consumo è più integrato, ma licensing e governance diventano centrali. |

## Lo scenario Qlik/Talend è forte quando contano analytics gestita, standardizzazione e time-to-start.

### Riuso

Alto se job, connettori, modelli Qlik e convenzioni di tenant sono trasferibili.

### Scalabilità

Buona, ma dipende da capacity, limiti tecnici e modello di licenza.

### Costi

Più subscription e capacity. Baseline utente: 50 GB tra 30k e 50k EUR.

### Blocker

Entitlement Talend/Qlik, lock-in e fit su file complessi.

Prima di usarlo come scenario finale serve una verifica contrattuale e tecnica sul tenant effettivo.

# La scelta definisce il modello operativo riusabile, non soltanto lo stack tecnologico.

|              | Scenario A<br>Dagster / dbt / Metabase               | Scenario B<br>Talend / Qlik                       |
|--------------|------------------------------------------------------|---------------------------------------------------|
| Riuso        | Asset, modelli e convenzioni tecniche.               | Job, connettori, tenant e modelli Qlik.           |
| Velocità     | Alta se si riusa CDG; media se serve setup completo. | Alta se licenze, tenant e connettori sono pronti. |
| Costi        | Infrastruttura, run e competenze.                    | Subscription, capacity e capability incluse.      |
| Complessità  | Engineering e operation cloud.                       | Licensing, governance prodotto e lock-in.         |
| Fit progetti | Forte su custom processing e reference CDG.          | Forte se Qlik è piattaforma target.               |

# ProSIGNAL e Kiron guidano la priorità; il CDG interno consolida la reference implementation.

Priorità alta

## ING ProSIGNAL

**Scenario A:** forte per file processing custom e controlli specifici.

**Scenario B:** da verificare su file grandi, fixed-column e controlli cross-file.

È il caso utile per stressare volumi, tracciati e controlli regolamentari.

Priorità alta

## Kiron CDG

**Scenario A:** coerente con un caso CDG-like documentato.

**Scenario B:** utile se Qlik è preferito come BI enterprise.

È il caso esterno più vicino alla logica CDG interno.

Reference

## CDG interno

**Scenario A:** molto coerente con il lavoro corrente.

**Scenario B:** possibile alternativa, ma implica ripensare parte della reference.

Serve come base per standard, controlli e convenzioni di delivery.

# Per i casi CDG la WBS segue il modello Kiron: processo al primo livello, funzioni al secondo.

## 1. PM e coordinamento

### FUNZIONI

Gestione progettuale

SAL interni e cliente

Raccolta open point

Allineamento piano e WBS

## 2. Actual

### FUNZIONI KIRON

Setup e analisi

Web app e base dati

Import Campus e Zucchetti

Configuratore regole

Ribaltamenti e data management

Output verso BI

## 3. Forecast

### FUNZIONI

Regole e parametri forecast

Motore calcolo forecast

Tabelle e viste forecast

Test interno

Rilascio e validazione

## 4. UAT e rilascio

### FUNZIONI

UAT cliente / owner CDG

Parallel run dove necessario

Deploy e validazione processo

Runbook e hypercare

Adattamento CDG interno: P1-P5 Actual e P6 Forecast

# ProSIGNAL usa la stessa logica WBS, ma i processi ruotano attorno a file, tracciati, controlli e output.

## 1. Intake e tracciati

### FUNZIONI

Upload e registro file

Gestione file grandi

Tracciati fixed-column

Versionamento campi

Mapping Lease / Conto Arancio

## 2. Parsing e validazione

### FUNZIONI

Parsing per range colonne

Controllo lunghezza record

Header e tail checks

Numero righe e formato

Gestione scarti e anomalie

## 3. Controlli funzionali

### FUNZIONI

Aggregazioni per forma tecnica

Quantità, importi e operazioni

Scostamento andamentale

Controlli incrociati file decadali

Audit regole applicate

## 4. Output e operatività

### FUNZIONI

Manipolazione e merge dati

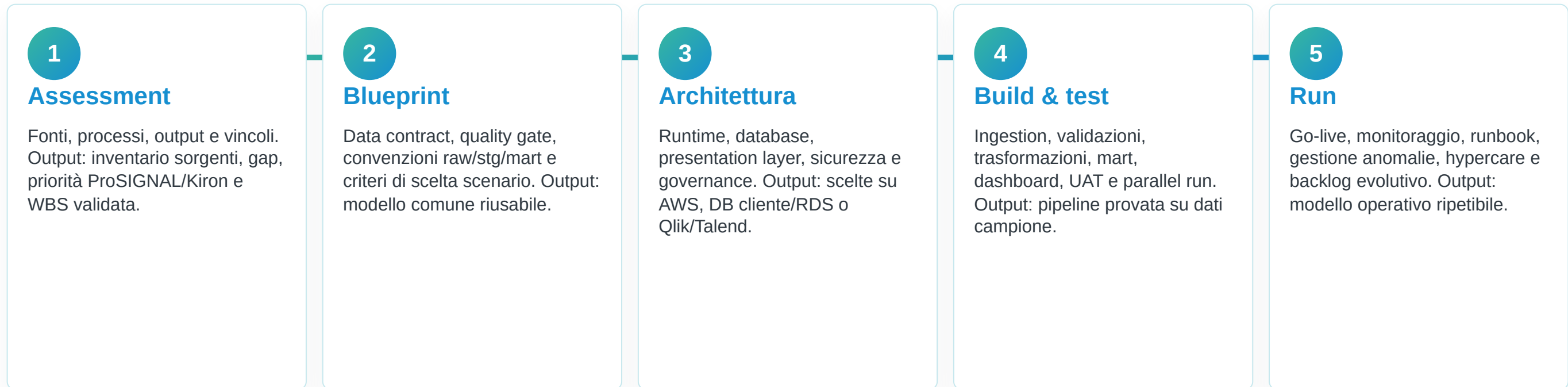
Export output regolamentari

Storicizzazione esiti

Dashboard controllo esiti

Log elaborazioni e workflow utente

# Il piano deriva dalla WBS: prima si validano processi e funzioni, poi si realizza per rilasci controllabili.



# Per una blueprint servono driver e range di discussione, non una quotazione definitiva.

## Setup blueprint

Analisi comune, convenzioni, template pipeline, criteri scenario e governance.

È lavoro comune: va progettato per non ripeterlo a ogni cliente.

## Implementazione

Adattamento per cliente: fonti, tracciati, regole, mart, dashboard e UAT.

Il costo dipende dalla distanza tra caso reale e blueprint.

## Run e licenze

Cloud/runtime, database, BI, monitoraggio, manutenzione ed evolutive.

Scenario B: ipotesi 50 GB tra 30k e 50k EUR, da confermare.



# La presentazione non chiude una scelta: mette sul tavolo le decisioni da prendere.

## Da decidere

- Quale modello operativo vogliamo standardizzare?
- Quanto lock-in prodotto è accettabile?
- Dove serve più custom processing?
- Quale scenario accelera ProSIGNAL e Kiron?

## Da verificare

- Entitlement Talend/Qlik e limiti capacity.
- Requisiti BI, accessi ed export per cliente.
- RDS o DB cliente per ciascun caso.
- Range economici completi di run e delivery.

